

CSS Flexbox

Master Flexible Box Layout for Modern Web Design

What is Flexbox?

Flexbox (Flexible Box Layout) is a CSS3 layout model that provides an efficient way to arrange, distribute, and align elements within a container. It's designed to handle both predictable and dynamic layouts with minimal code, making responsive design intuitive and powerful.

Key Advantages

- **One-dimensional layout:** Perfect for rows or columns
- **Flexible sizing:** Items grow and shrink automatically
- **Content-based alignment:** Easy centering and distribution
- **Order control:** Rearrange items without touching HTML
- **Responsive:** Adapts to container size automatically

Flex Container Properties

1. display: flex

Converts an element into a flex container, enabling flexbox layout for all direct children (flex items).

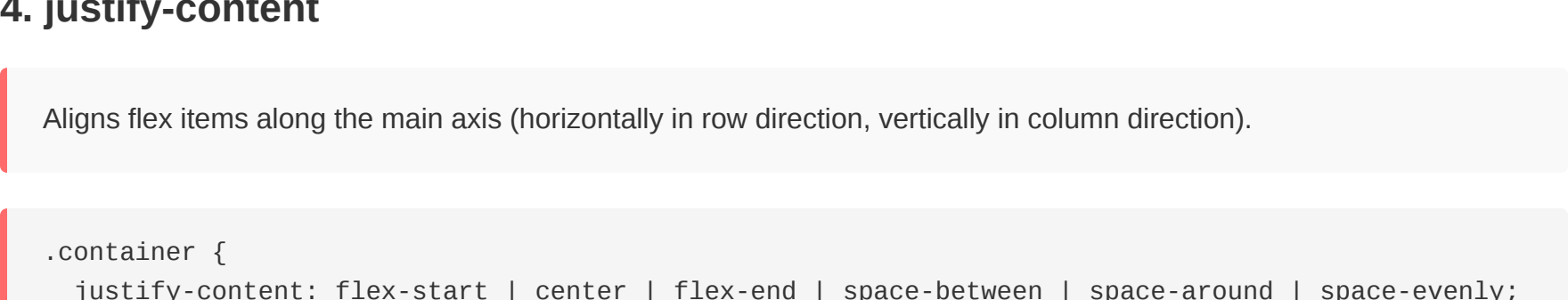
```
.container {
  display: flex;
}
```

2. flex-direction

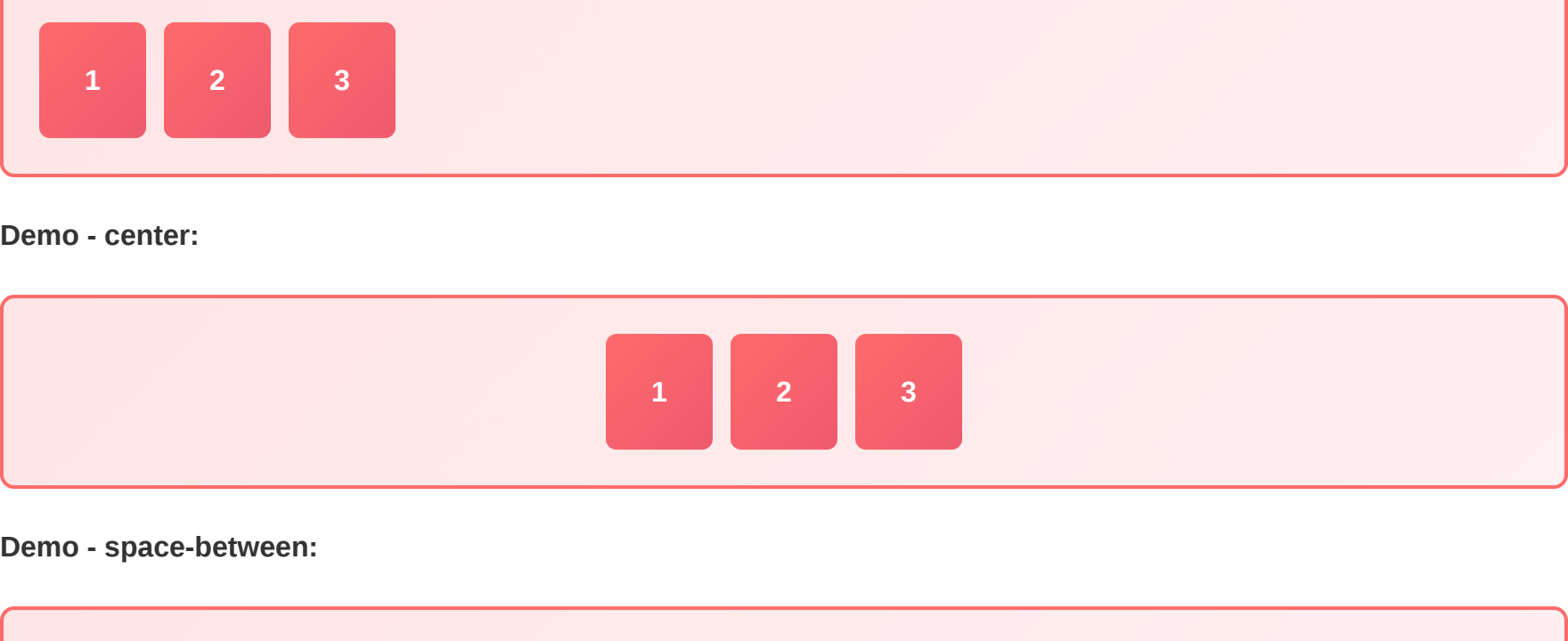
Defines the main axis direction for flex items. Default is **row** (horizontal).

```
.container {
  flex-direction: row | column | row-reverse | column-reverse;
}
```

Demo - Row:



Demo - Column:



3. flex-wrap

Controls whether flex items wrap to multiple lines when they don't fit in the container.

```
.container {
  flex-wrap: nowrap | wrap | wrap-reverse;
}
```

Demo - Wrap:

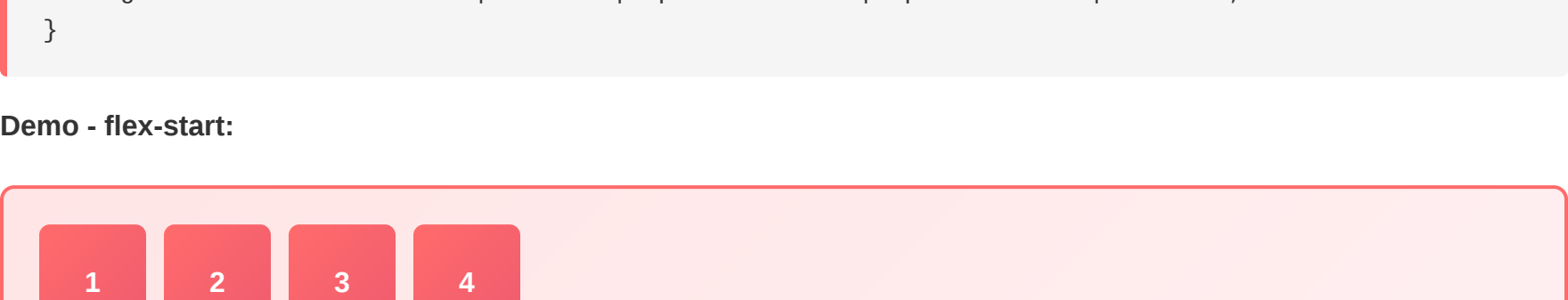


4. justify-content

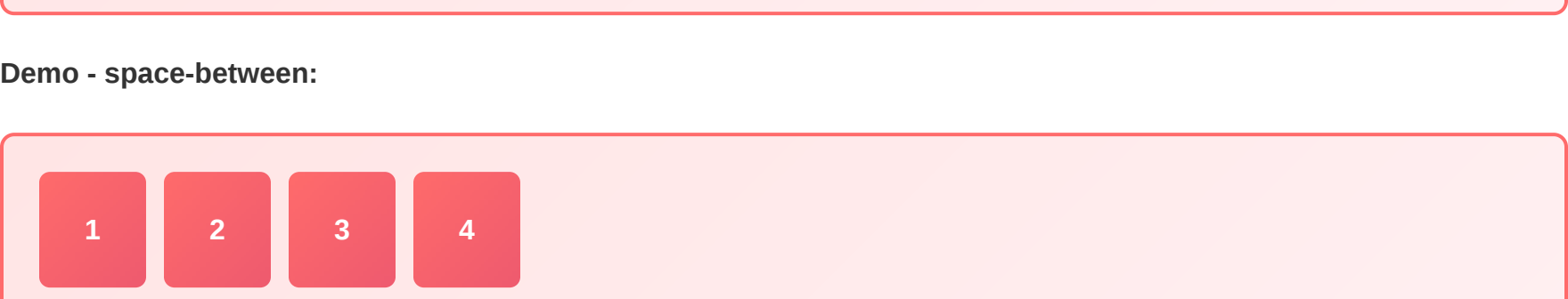
Aligns flex items along the main axis (horizontally in row direction, vertically in column direction).

```
.container {
  justify-content: flex-start | center | flex-end | space-between | space-around | space-evenly;
}
```

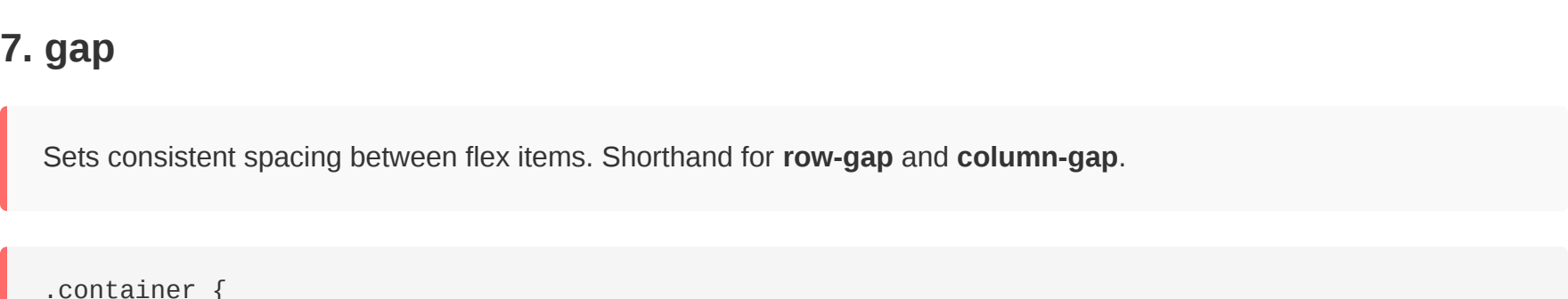
Demo - flex-start:



Demo - center:



Demo - space-between:



Demo - space-around:

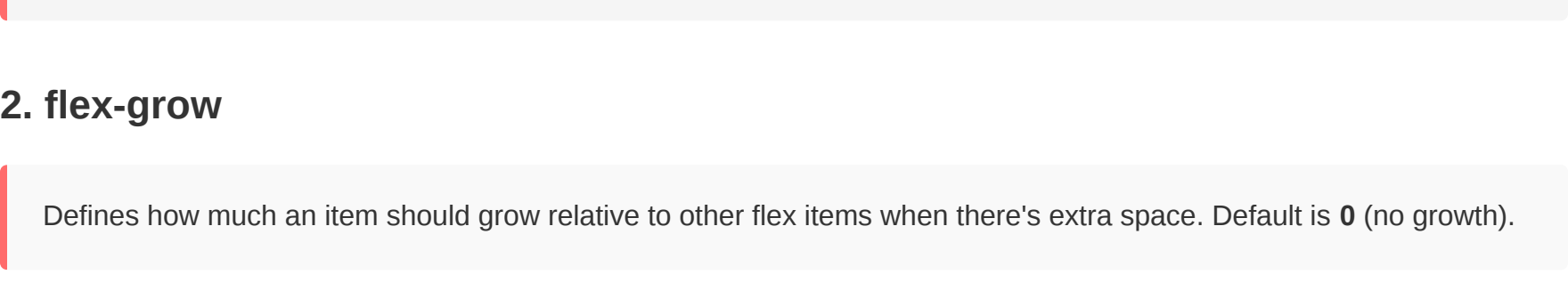


5. align-items

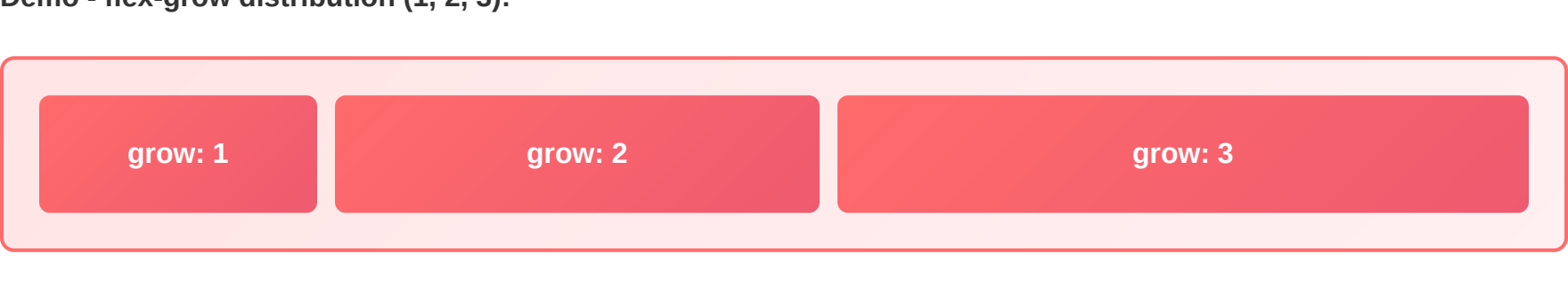
Aligns flex items along the cross axis (vertically in row direction, horizontally in column direction).

```
.container {
  align-items: flex-start | center | flex-end | stretch | baseline;
}
```

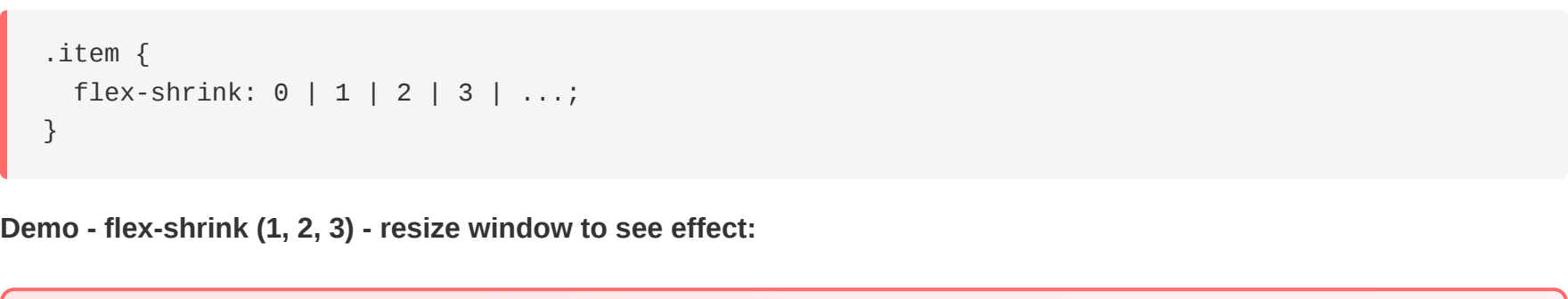
Demo - flex-start:



Demo - center:



Demo - stretch:

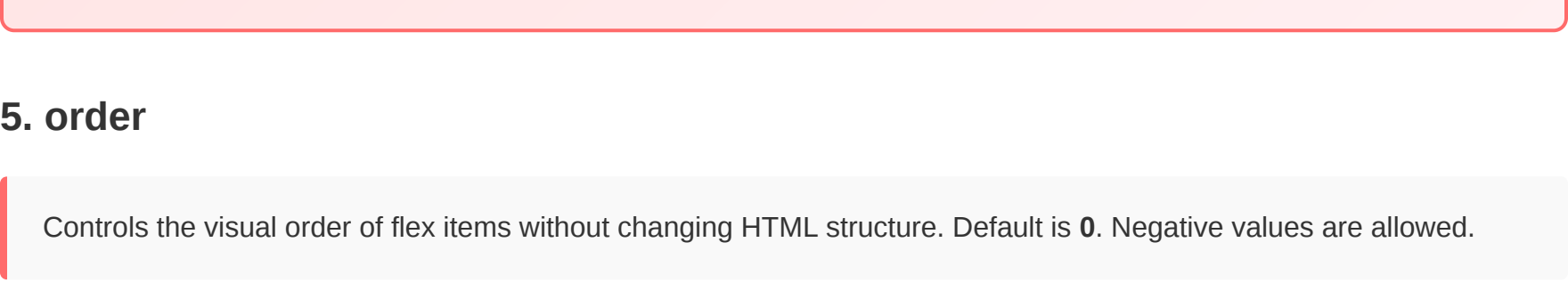


6. align-content

Aligns wrapped flex lines along the cross axis. Only works when **flex-wrap: wrap** is set.

```
.container {
  align-content: flex-start | center | space-between | space-around | stretch;
}
```

Demo - flex-start:



Demo - space-between:

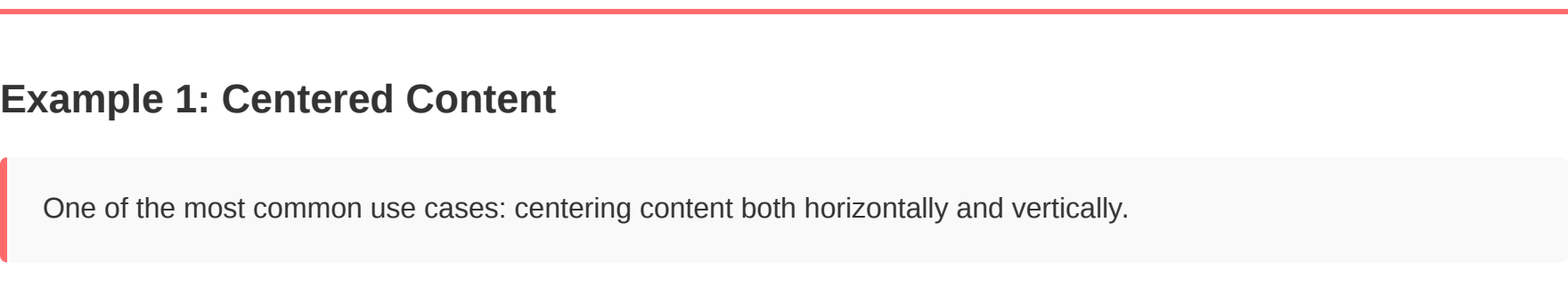


7. gap

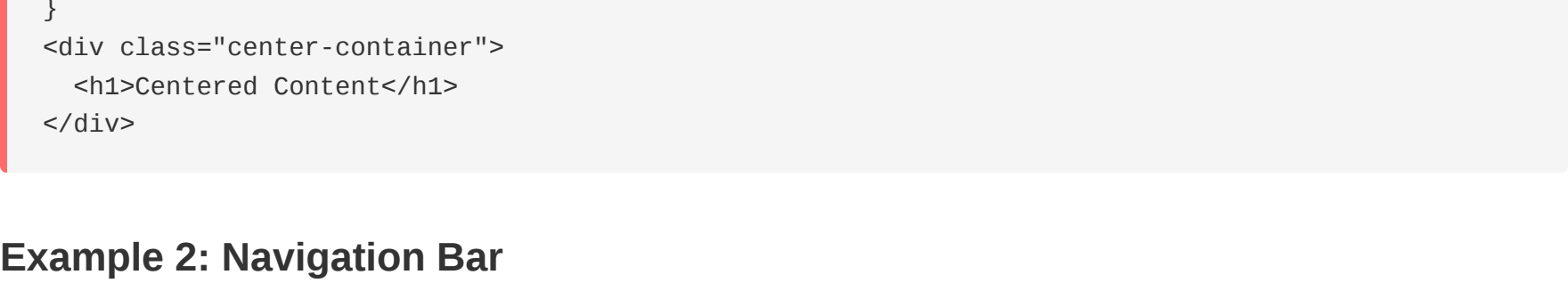
Sets consistent spacing between flex items. Shorthand for **row-gap** and **column-gap**.

```
.container {
  gap: 20px;
  /* or */ row-gap: 20px; column-gap: 10px;
}
```

Demo - gap: 10px:



Demo - gap: 30px:



Flex Item Properties

1. flex (Shorthand)

Shorthand for **flex-grow**, **flex-shrink**, and **flex-basis**. One of the most powerful flexbox properties.

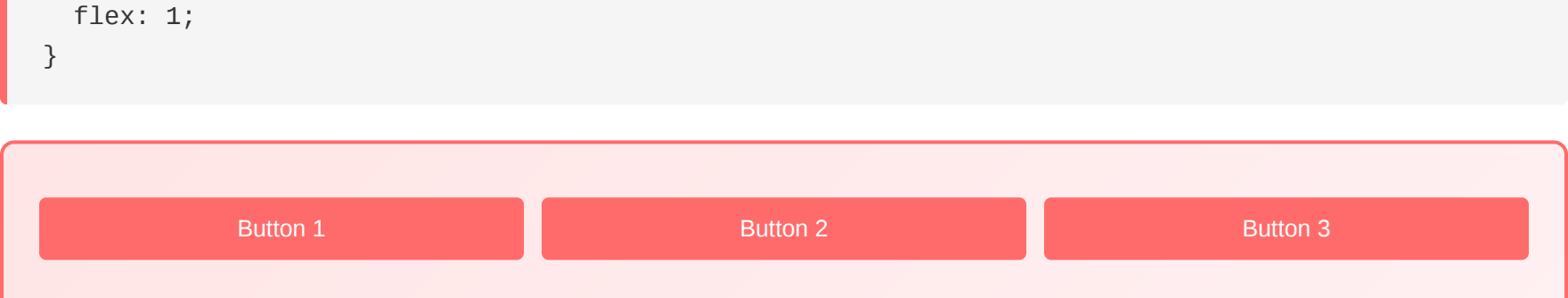
```
.item {
  flex: [flex-grow] [flex-shrink] [flex-basis];
  /* Default: 0 1 auto */
  /* Common: flex: 1; (means 1 1 0%) */
}
```

2. flex-grow

Defines how much an item should grow relative to other flex items when there's extra space. Default is **0** (no growth).

```
.item {
  flex-grow: 0 | 1 | 2 | 3 | ...;
}
```

Demo - flex-grow distribution (1, 2, 3):

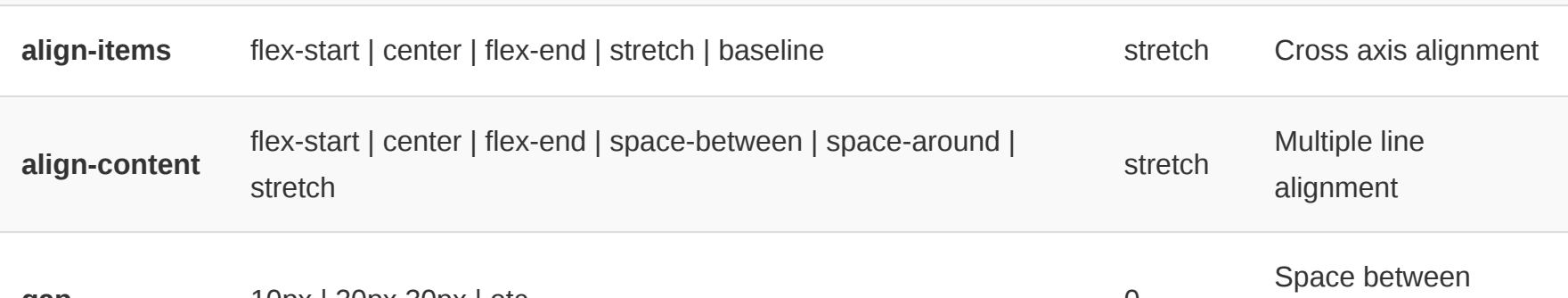


3. flex-shrink

Defines how much an item should shrink relative to others when space is limited. Default is **1**.

```
.item {
  flex-shrink: 0 | 1 | 2 | 3 | ...;
}
```

Demo - flex-shrink (1, 2, 3) - resize window to see effect:

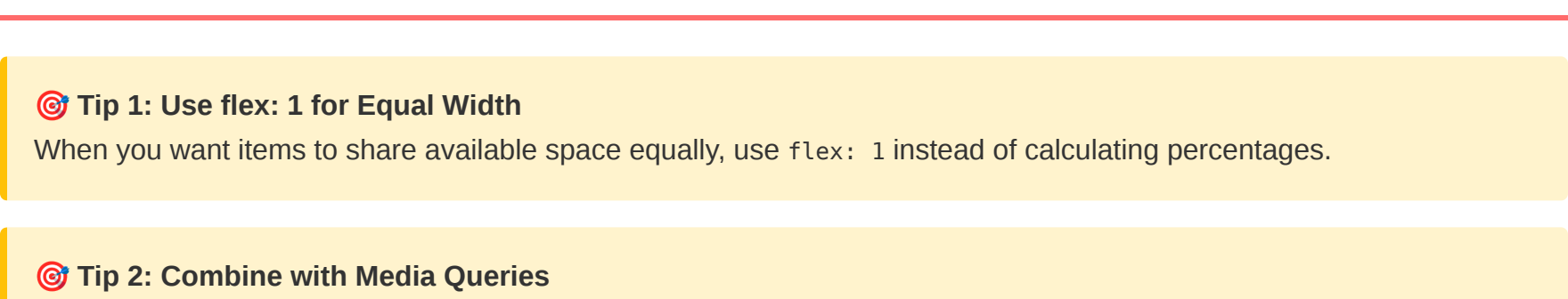


4. flex-basis

Defines the default size of a flex item before space is distributed. Acts like width in row direction or height in column direction.

```
.item {
  flex-basis: auto | 100px | 50% | 0 | ...;
}
```

Demo - flex-basis: 100px with flex-grow:

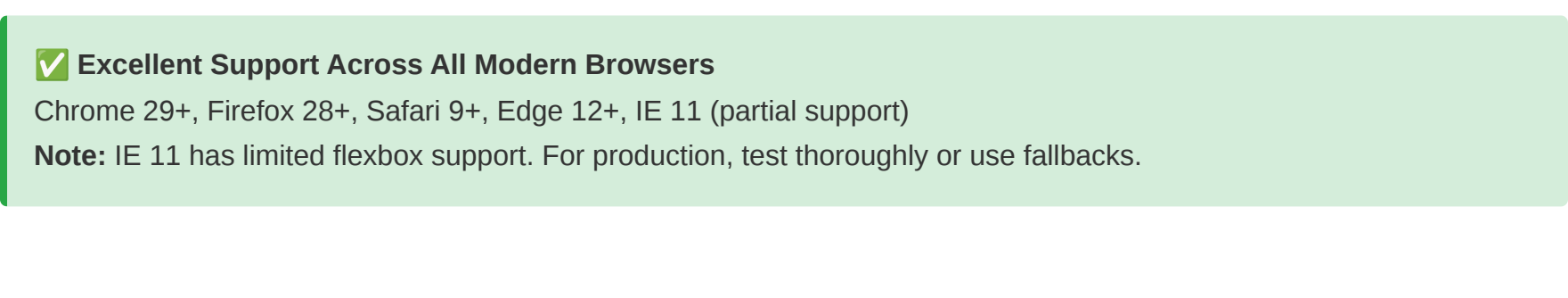


5. order

Controls the visual order of flex items without changing HTML structure. Default is **0**. Negative values are allowed.

```
.item {
  order: -1 | 0 | 1 | 2 | 999 | ...;
}
```

Demo - Items reordered to 3, 1, 2:



6. align-self

Overrides **align-items** for individual flex items. Allows per-item cross-axis alignment control.

```
.item {
  align-self: auto | flex-start | center | flex-end | stretch | baseline;
}
```

Demo - Different align-self values:



Practical Examples

Example 1: Centered Content

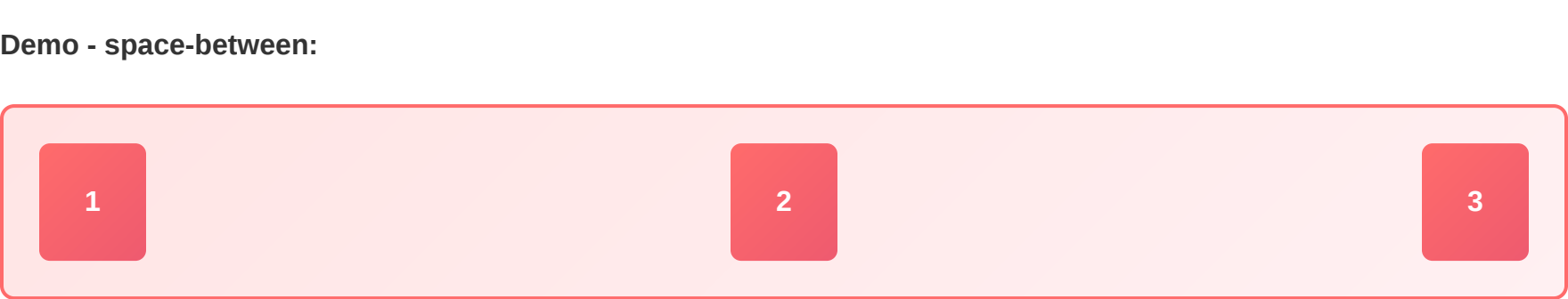
One of the most common use cases: centering content both horizontally and vertically.

```
.center-container {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 200px;
}
<div class="center-container">
  <h1>Centered Content</h1>
</div>
```

Example 2: Navigation Bar

Building a responsive navigation with logo, links, and authentication buttons.

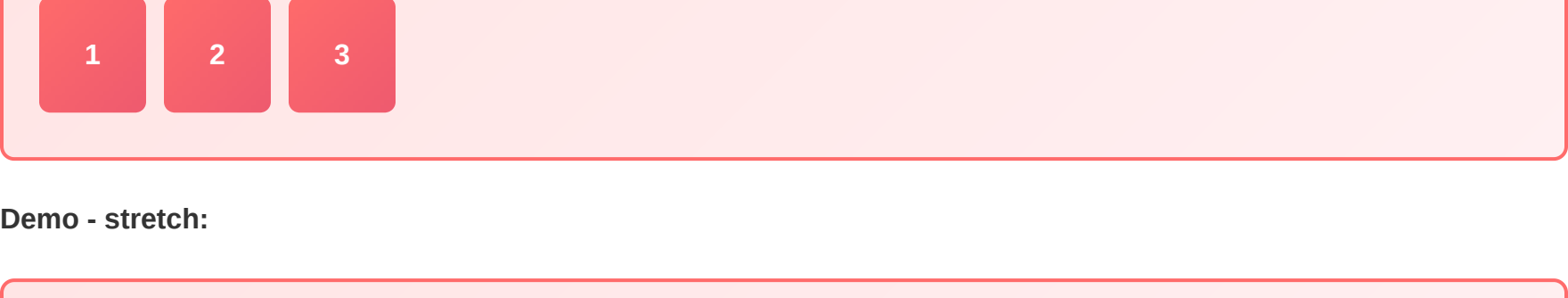
```
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 20px;
}
.navbar .links {
  display: flex;
  gap: 20px;
  flex: 1;
}
```



Example 3: Card Grid

Responsive grid layout that adapts from 1 to 3 columns automatically using flex-wrap and flex-basis.

```
.grid-container {
  display: flex;
  flex-wrap: wrap;
  gap: 15px;
}
.grid-item {
  flex: 1 1 calc(33.333% - 10px);
  min-width: 150px;
}
```



Example 4: Equal Width Buttons

Create buttons that stretch equally to fill available width.

```
.button-group {
  display: flex;
  gap: 10px;
}
.button-group button {
  flex: 1;
}
```



Complete Property Reference

Container Properties

Property	Values	Default	Purpose
display	flex inline-flex	—	Enables flexbox layout
flex-direction	row column row-reverse column-reverse	row	Main axis direction
flex-wrap	nowrap wrap wrap-reverse	nowrap	Line wrapping behavior
justify-content	flex-start center flex-end space-between space-around space-evenly	flex-start	Main axis alignment
align-items	flex-start center flex-end stretch baseline	stretch	Cross axis alignment
align-content	flex-start center flex-end space-between space-around stretch	stretch	Multiple line alignment
gap	10px 20px 30px etc.	0	Space between items

Item Properties

Property	Values	Default	Purpose
flex	1 0 1 auto 1 1 0% etc.	0 1 auto	Shorthand for grow, shrink, basis
flex-grow	0 1 2 3 ...	0	Growth factor
flex-shrink	0 1 2 3 ...	1	Shrink factor
flex-basis	auto 100px 50% 0 etc.	auto	Default item size
order	-1 0 1 2 ... 999	0	Visual order
align-self	auto flex-start center flex-end stretch baseline	auto	Individual alignment override

Best Practices & Tips

🔗 Tip 1: Use flex: 1 for Equal Width

When you want items to share available space equally, use `flex: 1` instead of calculating percentages.

🔗 Tip 2: Combine with Media Queries

Change `flex-direction` from row to column at mobile breakpoints for responsive layouts.

🔗 Tip 3: Use gap Instead of Margins

The `gap` property is cleaner and avoids margin-collapsing issues compared to margins on items.

🔗 Tip 4: Center Anything

For true centering, use `display: flex; justify-content: center; align-items: center;`

🔗 Tip 5: min-width on Flex Items

Without `min-width: 0`, flex items don't shrink below their content width.

🔗 Tip 6: flex-basis vs width

Use `flex-basis` instead of width on flex items for better flexibility and consistency.

Browser Support

✅ Excellent Support Across All Modern Browsers

Chrome 29+, Firefox 28+, Safari 9+, Edge 12+, IE 11 (partial support)

Note: IE 11 has limited flexbox support. For production, test thoroughly or use fallbacks.

Common Mistakes to Avoid

❌ Mistake 1: Forgetting to set flex-wrap when items overflow

✅ Fix: Use `flex-wrap: wrap` for items to flow to new lines.

❌ Mistake 2: Using flex items with explicit width and margin

✅ Fix: Let flex grow/shrink handle sizing, use gap for spacing.

❌ Mistake 3: Trying to align content with align-items on single-line flex

✅ Fix: Use align-content for multi-line, not align-items on wrapped items.

❌ Mistake 4: Not considering min-width for shrinking items

✅ Fix: Set `min-width: 0` on flex items if they contain text that needs to shrink.